

UNITY CATALOG · GA JULY 2026

ALL OR NOTHING

Wrap several SQL statements in one transaction. They commit together or they roll back together.

```
BEGIN ATOMIC ... END;
```

THE PROBLEM

THREE STATEMENTS, THREE SEPARATE COMMITS

1 `UPDATE accounts SET balance = balance - 100 WHERE id = 1;`
Ada is debited. Durable, on disk, irreversible.

COMMITTED

2 `UPDATE accounts SET balance = balance + 100 WHERE id = 2;`
Grace never gets the money.

FAILED

3 `INSERT INTO transfer_log VALUES (1, 2, 100, now());`
No audit trail of any of it.

NEVER RAN



£100 left one account and reached nobody. Your tables now disagree, and you repair them by hand.

Sign Up to the newsletter



databricks.news

Find more databricks tips @

dailydatabricks.tips

THE FIX

WRAP THEM AND THEY MOVE AS ONE

○ ○ ○

```
1 BEGIN ATOMIC
2   UPDATE accounts SET balance = balance - 100.00 WHERE account_id = 1;
3   UPDATE accounts SET balance = balance + 100.00 WHERE account_id = 2;
4   INSERT INTO transfer_log
5     VALUES (1, 2, 100.00, current_timestamp());
6 END;
7
8 -- Two tables. One commit. One Delta log entry.
```

 UC MANAGED TABLES

 CATALOG COMMITS ON

 DBR 18.0+ OR SQL WAREHOUSE

Sign Up to the newsletter

 [databricks.news](#)

Find more databricks tips @

[dailydatabricks.tips](#)

THE PROOF

THE LAST STATEMENT FAILS. WATCH THE FIRST TWO VANISH.

○ ○ ○

```
1 BEGIN ATOMIC
2   -- These two succeed...
3   INSERT INTO transfer_log VALUES (2, 1, 5000.00, now());
4   UPDATE accounts SET balance = balance + 5000.00 WHERE account_id = 1;
5
6   -- ...then this trips CHECK (balance ≥ 0)
7   UPDATE accounts SET balance = balance - 5000.00 WHERE account_id = 2;
8 END;
```

ADA'S BALANCE
400.00
Credit undone

LOG ROWS
1
Insert undone

DELTA COMMITS
0
Nothing written

Sign Up to the newsletter



Find more databricks tips @

dailydatabricks.tips

THE RULES

Sign Up to the newsletter

 [databricks.news](#)

Find more databricks tips @

[dailydatabricks.tips](#)

SIX THINGS TO GET RIGHT



Turn on catalog commits

`delta.feature.catalogManaged = 'supported'` on every table you write to.



No DDL inside

CREATE, ALTER and DROP run outside the transaction. So does time travel.



Prefer BEGIN ATOMIC

Row-level conflict detection and automatic rollback. No open session to leak.



Build retry logic

Commits are optimistic. Conflicts surface at commit time and the loser fails.



Reads are repeatable

First touch pins a snapshot. Later reads ignore everyone else's commits.



Know the ceilings

100 tables per transaction. Everything rolls back after 48 hours.



STOP REPAIRING HALF-WRITTEN DATA

`BEGIN ATOMIC ... END;`

Full walkthrough → dailydatabricks.tips/s/transactions

SIGN UP TO THE NEWSLETTER

 [Databricks.news](https://databricks.news)

dailydatabricks.tips